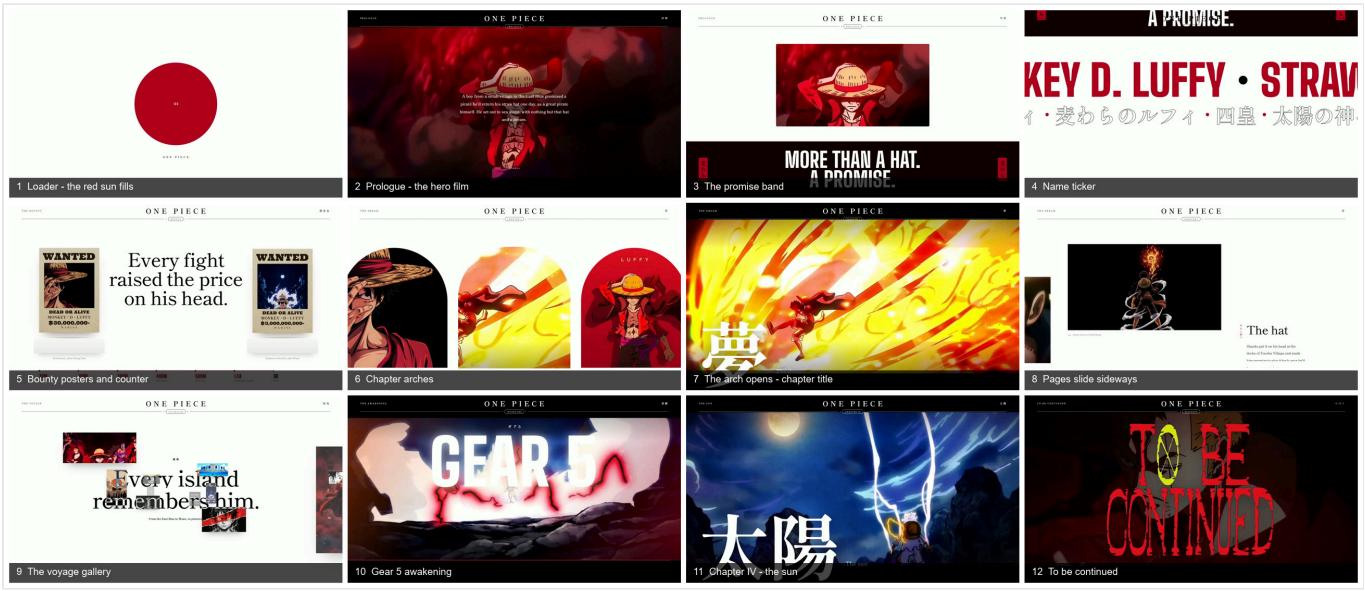
ONE PIECE | Luffy: How to Recreate the Website

October 2, 2026

This guide rebuilds the ONE PIECE | Luffy fan website from nothing: a white, cinematic scroll site where Luffy's story plays out in four chapters, with slow-motion footage, arch-window reveals and a Gear 5 awakening you scrub through by scrolling. Follow the steps in order; each one says what to do, which tool does it, and what you should see.

What you are building

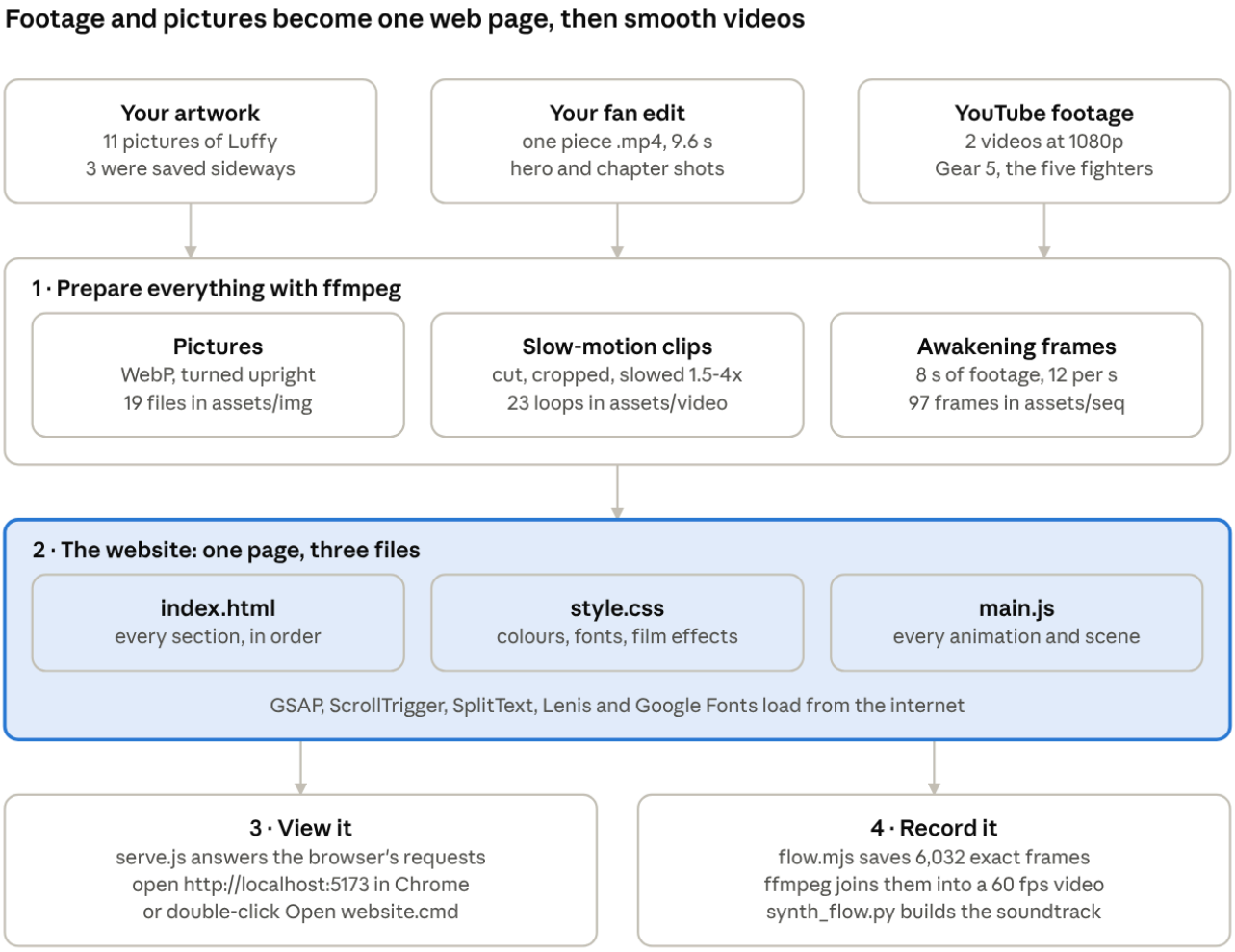
One long page in white, black and Luffy crimson that plays like a film as you scroll. There are no links to click: pinned scenes hold the screen while their animation plays, then let go, and the visitor's scroll sets the speed.



Page order: Loader → Prologue → Name ticker → Bounty → Chapter I, The dream → The voyage gallery → Chapter II, The crew → Chapter III, The will → The awakening → Chapter IV, The sun → Epilogue → Footer.

The finished project has 23 slow-motion clips, 19 images and a 97-frame scroll sequence, about 45 MB in total.

How it fits together



Read it top to bottom: the raw files are prepared once with ffmpeg, the website only ever uses those prepared copies, and the finished page is either opened in Chrome or recorded into videos.

What you need

Everything is free. Install the tools once; the libraries and fonts load from the internet, so there is nothing else to install.

Item	What it does in this project	Where to get it
A code editor (VS Code)	Edit the HTML, CSS and JavaScript files	code.visualstudio.com
Node.js 20 or newer	Runs the local server and the video recorder	nodejs.org
ffmpeg	Converts images, cuts and slows the footage, encodes the videos	ffmpeg.org
Google Chrome	Viewing the site; the recorder drives it to make the videos	google.com/chrome

Item	What it does in this project	Where to get it
yt-dlp (optional)	Downloads reference videos you are allowed to use	github.com/yt-dlp/yt-dlp
Python 3 with numpy and scipy (optional)	Builds the soundtrack for the videos	python.org
GSAP 3.15, with ScrollTrigger and SplitText	All animation, the scroll-driven scenes and the line-by-line text reveals	Loaded from cdn.jsdelivr.net/npm/gsap@3.15.0
Lenis 1.3.26	Smooth, weighted scrolling	Loaded from cdn.jsdelivr.net/npm/lenis@1.3.26
Shippori Mincho B1, Big Shoulders Display, Ultra	The three typefaces: serif and Japanese, tall condensed titles, wanted-poster numbers	Loaded from fonts.google.com

You also need the raw material: Luffy artwork (the project used 11 images) and footage (a short fan edit plus two YouTube videos).

Step 1 · Set up the project folder

Make one project folder with the website inside it. Keep your original images and footage at the top level and never edit them: every file the site uses is a converted copy inside `website/assets`.

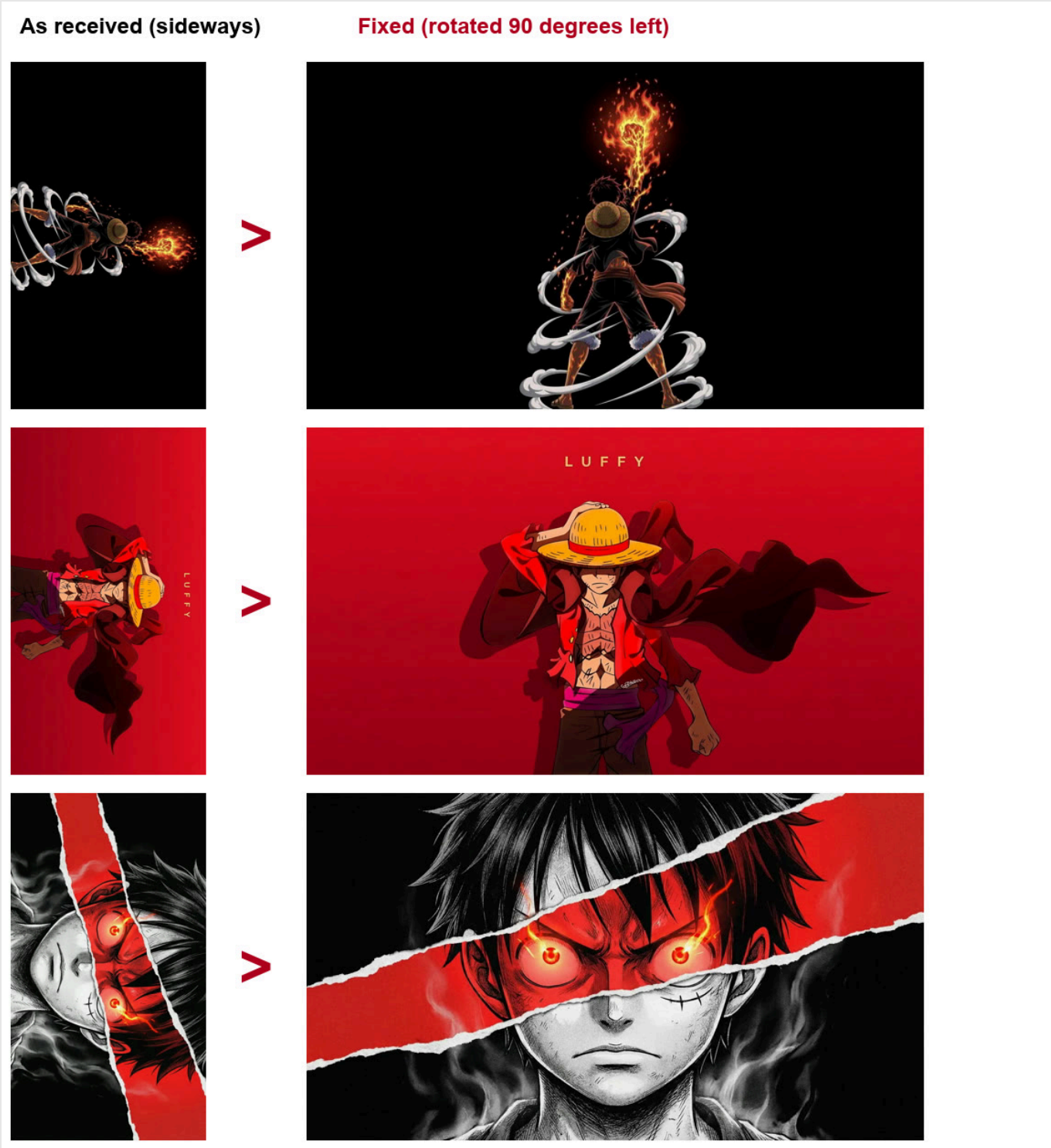
```
one piece/
├── (your original images and footage)
├── website/                the site itself
│   ├── index.html          every section, in order
│   ├── css/style.css       colours, fonts, layout, film effects
│   ├── js/main.js          every animation and scroll scene
│   ├── serve.js            tiny local server (the videos need it)
│   ├── Open website.cmd    double-click to run the site
│   └── assets/
│       ├── img/            19 pictures (.webp)
│       ├── video/          23 slow-motion clips (.mp4) and their posters (.webp)
│       └── seq/awaken/     97 frames for the Gear 5 scroll sequence
├── source/youtube/         downloaded reference footage
├── launch-video/           the 40-second launch film and its scripts
└── website-flow/           the 100-second screen recording and its recorder
```

If the folder lives in OneDrive or Dropbox, keep large working files (thousands of video frames, `node_modules`) outside it so syncing does not slow everything down.

Step 2 · Prepare the images

Convert every picture to WebP (same look, about five times smaller) and check that each one is upright. Three of the original pictures were landscape artwork saved sideways from a phone; they had to be turned 90

degrees to the left.



How to spot a sideways picture: text runs down the side, the eyes are stacked one above the other, or a person seems to be lying down. Luffy's scar sits under his left eye, which is on your right when he faces you.

```
# convert a picture to WebP
ffmpeg -i original.jpg -c:v libwebp -quality 86 website/assets/img/luffy-hat.webp

# fix a sideways picture: transpose=2 turns it 90 degrees left, transpose=1 turns it right
ffmpeg -i sideways.jpg -vf transpose=2 -c:v libwebp -quality 86 website/assets/img/red-hawk.webp

# crop away a strip, here the top 60 pixels where a watermark sat
ffmpeg -i embers.jpg -vf "crop=iw:ih-60:0:60" -c:v libwebp -quality 86 website/assets/img/embers.webp

# take a still from footage, cropped above burned-in subtitles
ffmpeg -ss 33.0 -i amv.mp4 -frames:v 1 -vf "crop=iw*0.96:iw*0.96/2.39:iw*0.02:ih*0.04,scale=1600:-2" -c:v
libwebp -quality 84 website/assets/img/s-five.webp

# cut out the TO BE CONTINUED card by making its black background transparent
ffmpeg -ss 15.5 -i luffy-clip.mp4 -frames:v 1 -vf
"crop=iw*0.74:ih*0.98:iw*0.13:ih*0.01,format=rgba,colorkey=0x000000:0.22:0.12" tbc.png
```

File	What it shows	Where it appears
luffy-hat.webp	Luffy tipping his hat over his face	Chapter I arch, first wanted poster, gallery
luffy-red.webp	Luffy holding his hat on red, "LUFFY" above (was sideways)	Chapter I arch, gallery
red-hawk.webp	Luffy from behind, raising a flaming fist (was sideways)	Chapter I page, gallery
silhouette.webp	Luffy's silhouette under the straw hat	Chapter I last page, gallery
crew-sunset.webp	The crew watching the sunset	Chapter II arch, gallery
nakama.webp	The nine Straw Hats in coloured strips	Chapter II page, gallery
jolly-rogers.webp	Every Straw Hat's Jolly Roger, stacked	Chapter II crew list, gallery
embers.webp	Luffy in his red coat among embers	Chapter III arch, gallery
rage-eyes.webp	Luffy's face torn between ink and red (was sideways)	Chapter III arch, gallery
gear5.webp	Gear 5 Luffy laughing under the moon	Second wanted poster, Chapter IV arch, gallery
tbc.webp	The TO BE CONTINUED card, background removed	The ending
s-*.webp (8 files)	Stills taken from the footage	Gallery cards, Chapter II and IV arches

Step 3 · Turn the footage into slow-motion loops

Every clip on the site is one short shot: cropped to hide subtitles, slowed down 1.5 to 4 times with newly drawn in-between frames, and looped so it never jumps.



1. **Find the shots.** List every cut in a video (first command below), then make a contact sheet (a grid of one frame per shot) and pick the strongest moments.
2. **Crop away subtitles.** Fan edits often have subtitles burned into the bottom 10% of the picture. Cut a wide cinema-shaped band (2.39:1) from above them: `crop=iw*0.96:iw*0.96/2.39:iw*0.02:ih*0.04`.
3. **Slow it down smoothly.** Anime holds each drawing for two or three frames. `mpdecimate` drops the repeats, `setpts=2*(PTS-STARTPTS)` stretches time two times, and `minterpolate` paints new in-between frames from the motion, at 60 frames per second.
4. **Make it loop.** Gentle motion (a coat billowing, a head turning) plays forward then backward. Motion in one direction (a ship sailing, an elephant walking) instead fades its last 0.6 seconds into its first.
5. **Save it for the web.** H.264 MP4 with no sound, plus a poster picture the page shows before the clip starts.

```
# list every cut in a video
ffmpeg -i amv.mp4 -vf "scale=480:-2,select='gt(scene,0.32)',showinfo" -an -f null - 2>&1 | grep pts_time
```

The commands for one clip, the five fighters on the roof:

```
# 1. cut, crop, scale, then slow down 2x with new in-between frames at 60 fps
ffmpeg -ss 31.70 -to 34.20 -i amv.mp4 -an -vf
"crop=iw*0.96:iw*0.96/2.39:iw*0.02:ih*0.04,scale=1280:536,mpdecimate,setpts=2*(PTS-
STARTPTS),minterpolate=fps=60:mi_mode=mci:mc_mode=aobmc:me_mode=bidir:vsbmc=1,eq=contrast=1.05:saturation=1.
07:gamma=0.98" -c:v libx264 -crf 14 -preset fast forward.mp4

# 2. ping-pong loop: forward, then backward (N = the number of frames in forward.mp4)
ffmpeg -i forward.mp4 -filter_complex "[0]split[f][b];[b]reverse,trim=start_frame=1:end_frame=N-
1,setpts=PTS-STARTPTS[r];[f][r]concat=n=2:v=1" -c:v libx264 -crf 21 -preset slow -pix_fmt yuv420p -movflags
+faststart -an website/assets/video/five.mp4

# 3. poster picture, taken from the middle of the loop
ffmpeg -ss 3.6 -i website/assets/video/five.mp4 -frames:v 1 -c:v libwebp -quality 80
website/assets/video/five.webp
```

The Gear 5 awakening is not a video but 97 still frames, so scrolling can move through it in either direction:

```
ffmpeg -ss 152.20 -to 160.20 -i amv.mp4 -vf "fps=12,crop=iw*0.96:ih*0.96:iw*0.02:ih*0.02,scale=1280:720" -
c:v libwebp -quality 68 website/assets/seq/awaken/%03d.webp
```

Before you keep a clip, look at its first and last frames: fan edits hide quick flashes and dissolves between shots, and a loop that starts on one blinks every few seconds. Slow motion is slow to make, about a minute per second of footage at 1280 pixels wide, so run several clips at once.

All 23 clips, for an exact rebuild (sources: A = the fan edit "one piece .mp4", B = the Joy Boy AMV, C = the "I'm Monkey D Luffy" clip):

Clip	Source and seconds	Slower	Loop	Size	Used in
hero	A 0.02–0.50	4x	ping-pong	1920×1080	Prologue
flames	A 8.27–9.04	3x	plain	1920×1080	Chapter I opener
declare	C 12.85–14.30	2x	ping-pong	1280×720	Chapter I
young	B 144.85–146.85	2x	ping-pong	1280×720	Chapter I
hattip	C 8.47–10.75	2x	ping-pong	720×900	Chapter I
lineup	A 0.62–2.20	2x	ping-pong	1920×1080	Chapter II opener
waves	B 60.12–61.46	2x	crossfade	1280×536	Chapter II
zoro	A 3.10–3.90	2.5x	ping-pong	1920×1080	Chapter II
snowship	B 55.70–58.60	1.5x	crossfade	1280×536	Chapter II
coat	A 6.54–7.33	3x	ping-pong	1920×1080	Chapter III opener
five	B 31.70–34.20	2x	ping-pong	1280×536	Chapter III
kaido	A 9.07–9.57	3x	ping-pong	1920×1080	Chapter III
standoff	C 0.00–2.40	2x	ping-pong	1280×536	Chapter III
sky	A 2.30–3.07	2.5x	ping-pong	1280×720	Chapter III
law	A 3.93–4.77	2.5x	ping-pong	960×540	Chapter III
yamato	A 4.80–5.67	2.5x	ping-pong	960×540	Chapter III
kaido-blue	A 7.37–8.23	2.5x	ping-pong	960×540	Chapter III
g5moon	B 168.24–168.86	3x	ping-pong	1920×1080	Chapter IV opener
zunesha	B 147.00–150.60	1.5x	crossfade	1280×536	Chapter IV
g5run	B 177.99–179.24	2.5x	ping-pong	1280×720	Chapter IV
nika	B 163.12–163.70	3x	ping-pong	1280×536	Chapter IV
dragon	B 201.78–202.48	3x	ping-pong	1280×536	Chapter IV
reel	A 0–9.58	none	plain	1920×1080	Ending

Step 4 · Build the page

The whole site is one HTML file: a loader, a fixed header, eleven sections stacked in scroll order, and a footer. Each section below is a block of plain HTML; Step 6 brings it to life.

#	Section	What the visitor sees	How it works
1	Loader	A red sun fills as the files load, then opens like a lens onto the first film	A counter plus a growing circle mask
2	Header (always on top)	The ONE PIECE wordmark, the chapter name in English and Japanese, a small pill	Fixed bar; its words roll when the chapter changes; its ink flips white over film
3	Prologue	The hero film with the opening line; the film shrinks into a frame and the "More than a hat. A promise." band rises	A pinned scene whose animation follows the scroll
4	Name ticker	Two rows of giant names drifting sideways, faster while you scroll	A looping strip whose speed follows scroll velocity
5	Bounty	Two wanted posters rise; the bounty climbs from ฿30,000,000 to ฿3,000,000,000 along a timeline of his arcs	Pinned scene with a counting number
6	Chapter I, The dream	Three arch windows; the middle one opens into a full-screen film; the title 夢 slides in; then pages of text, films and pictures slide sideways	Pinned scene, a reshaping mask, a sideways track
7	The voyage	Sixteen pictures fly past the camera in 3D around "Every island remembers him."	Pinned scene, pictures moved in depth
8	Chapter II, The crew	Same chapter pattern: the crew list, the Nakama poster, the ships	As section 6
9	Chapter III, The will	Same pattern: Onigashima, the five fighters, Conqueror's Haki	As section 6
10	The awakening	Scrolling plays Gear 5's awakening frame by frame, with film subtitles, then GEAR 5 slams in	97 frames drawn on a canvas
11	Chapter IV, The sun	Same pattern: the drums of liberation, Gear 5, Nika, the dragon	As section 6
12	Epilogue	"The One Piece is still out there.", the final film rises, TO BE CONTINUED slams in	Pinned scene, curtain mask, a camera shake
13	Footer	A Start again button and the fan-use note	Plain HTML

The skeleton of `index.html` (shortened; every chapter repeats the same pattern):

```
<div class="loader">…red sun and counter…</div>
<header class="hud">…wordmark, chapter name, pill…</header>
<main>
  <section class="prologue scene" data-label="Prologue" data-jp="序章">…</section>
  <section class="marquee">…two rows of names…</section>
  <section class="wanted scene">…posters, bounty timeline…</section>
  <section class="chapter scene" id="dream" data-label="The dream" data-jp="夢">
    <div class="chapter__pin"><div class="chapter__track">
      <div class="panel panel--opener">two arch pictures, the film, the kanji title</div>
      <article class="panel panel--text">…a heading and a paragraph…</article>
      <figure class="panel panel--film">…a slow-motion clip and its caption…</figure>
    </div></div>
  </section>
  …the voyage, three more chapters, the awakening, the epilogue…
</main>
<footer class="foot">…</footer>
```

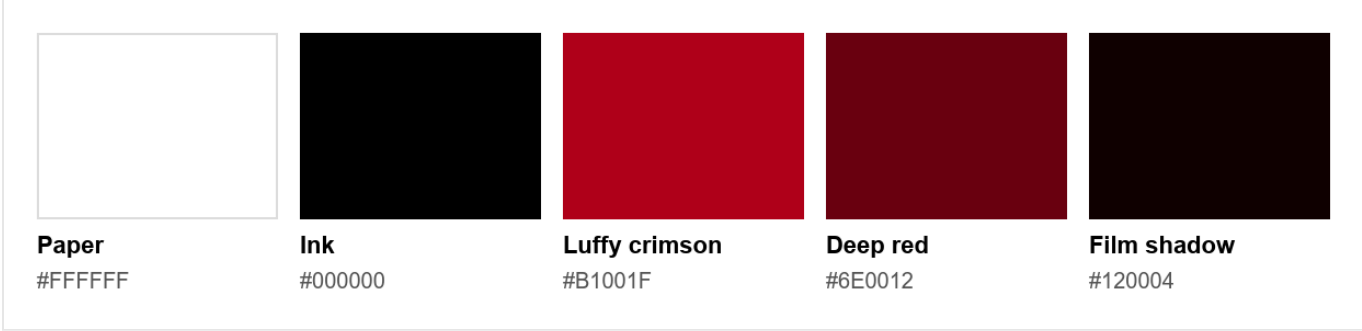
Every clip uses the same tag, so it downloads only when it is about to appear and shows its poster until then:

```
<video data-src="assets/video/flames.mp4" poster="assets/video/flames.webp"
       muted loop playsinline preload="none"></video>
```

The `data-label` and `data-jp` words on each section are what the header shows while that section fills the screen.

Step 5 · The look

White paper, black ink and one red: the page stays quiet so the footage carries the colour. The crimson was sampled from the footage itself, so the page and the films match.



Typeface	Used for	Weights
Shippori Mincho B1	The ONE PIECE wordmark, headings, body text and every Japanese word	400 to 800
Big Shoulders Display	Tall condensed titles: the promise band, the name ticker, GEAR 5	600 to 900
Ultra	The numbers on the wanted posters and the bounty	400

Four film effects make the footage feel like cinema, all done in CSS on top of each video:

- **Grade:** darkened edges, and a warm orange to crimson tint laid across with a colour-burn blend.
- **Grain:** a noise texture the page draws itself, shifted six times a second.
- **Letterbox:** black bars that grow in from the top and bottom on big moments.
- **Burn:** a glowing orange-white light leak swept across the frame as an arch opens.

```
:root {
  --white: #ffffff;  --ink: #000000;  --red: #b1001f;  --red-deep: #6e0012;  --blood: #120004;
  --serif: "Shippori Mincho B1", serif;
  --cond: "Big Shoulders Display", sans-serif;
  --poster: "Ultra", serif;
}

/* an arch window: a tall box whose top corners are half its width */
.arch { width: 32vw; top: 24vh; bottom: 12vh; border-radius: 16vw 16vw 0 0; overflow: hidden; }

/* the film grade: dark edges, then a warm-to-red colour burn */
.film__grade {
  background: radial-gradient(120% 95% at 50% 50%, transparent 52%, rgba(18,0,4,.62) 100%),
              linear-gradient(180deg, rgba(18,0,4,.5), transparent 24%, transparent 74%, rgba(18,0,4,.55));
}
.film__grade::after {
  content: ""; position: absolute; inset: 0; mix-blend-mode: color-burn;
  background: linear-gradient(118deg, rgba(255,92,20,.5), transparent 34%, transparent 62%,
rgba(177,0,31,.55));
}
```

On screens narrower than 820 pixels, chapters stack downwards instead of sliding sideways and the side arches are hidden. If a visitor's device asks for reduced motion, the page shows everything still, with posters in place of moving video.

Step 6 · The motion

Scrolling is the timeline. Each scene is a GSAP animation tied to the scroll position: scroll forward and it plays, scroll back and it rewinds. All of it lives in `js/main.js`.

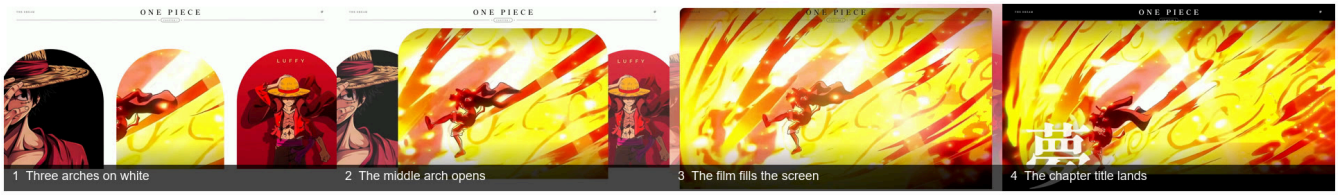
1. Smooth scrolling. Lenis turns the mouse wheel's steps into one smooth glide. It runs on GSAP's clock, so the scroll and the animations never drift apart.

```
const lenis = new Lenis({ lerp: 0.075, wheelMultiplier: 0.85 });
lenis.on('scroll', ScrollTrigger.update);
gsap.ticker.add(time => lenis.raf(time * 1000));
```

2. Pinned scenes. `pin` holds a section on screen for a set scroll distance while its animation plays; `scrub` ties the animation to the scroll, with 0.8 seconds of smoothing.

```
const tl = gsap.timeline({
  scrollTrigger: { trigger: '.prologue', start: 'top top', end: '+=240%', pin: true, scrub: 0.8 },
});
tl.to('.prologue__intro', { opacity: 0, duration: 0.18 }, 0.04)
  .fromTo('.prologue__film', { clipPath: 'inset(0% 0% 0% 0% round 0px)' },
                             { clipPath: 'inset(18% 27% 38% 27% round 6px)', duration: 0.4 }, 0.24);
```

3. The arch reveal. Each chapter's film starts cut to the shape of the middle arch, and the cut grows until the film fills the screen. The side arches slide away, a light burn sweeps across, black bars close in, and the kanji title slides into place.




```
// from the arch shape to the full screen, in pixels at both ends so the change is smooth
tl.fromTo(film,
  { clipPath: `inset(${top}px ${side}px ${bottom}px ${side}px round ${r}px ${r}px 0px 0px)` },
  { clipPath: 'inset(0px 0px 0px 0px round 0px 0px 0px 0px)', duration: OPEN * 0.55 }, 0);
```

4. Chapters slide sideways. After the arch opens, all of a chapter's pages sit side by side on one long track, which moves left by its extra width. One pixel of scroll moves it one pixel, and the pin lasts exactly as long as the animation. As each page arrives, its text rises line by line (SplitText cuts paragraphs into masked lines) and its picture opens like a curtain.

```
const slide = track.scrollWidth - innerWidth;
tl.to(track, { x: -slide, duration: slide }, OPEN);
```

5. The awakening. The 97 frames load as pictures. The scroll position picks a frame (progress × 96) and draws it on a canvas, with the next frame laid over it, partly transparent, so a slow scroll glides instead of stepping.



```
const f = progress * (frames.length - 1);
const i = Math.floor(f);
ctx.globalAlpha = 1;      ctx.drawImage(frames[i], ...rect);
ctx.globalAlpha = f - i;  ctx.drawImage(frames[i + 1], ...rect);
```

6. The voyage. Each of the 16 pictures is given a depth. As you scroll, they all move toward the camera, and CSS perspective makes the near ones larger. Each fades in out of the white distance and fades out just before it reaches the lens.

7. The name ticker. Each row holds two copies of its names side by side. A small loop shifts them every frame and wraps around; scroll speed is added to the drift, and scrolling up reverses it.

8. Video. A clip downloads only when it is about to appear, plays only while it is on screen, and pauses when it leaves, which keeps the page light.

9. The header. Each scene says whether the header's ink should be white (over film) or black (over paper), and the chapter name rolls up whenever a new scene takes the screen.

Step 7 · Run it on your computer

Start the small local server, then open http://localhost:5173 in Chrome. On Windows you can simply double-click `Open website.cmd` in the website folder, which does both; close its black window to stop it.

```
cd website
node serve.js
```

The server matters because of the videos. To loop and jump inside a clip, the browser asks for pieces of the file (a "range request"), and `serve.js` answers those; opening `index.html` straight from the folder can leave clips frozen or unable to loop. The server is about 60 lines of plain Node.js with nothing to install.

Check the result three ways before calling it done:

- ☐ A desktop window about 1440 × 900: chapters slide sideways and every clip plays.
- ☐ A phone-width window (in Chrome's device toolbar, 390 × 844): chapters stack downwards and nothing spills sideways.
- ☐ Reduced motion switched on in your system settings: the page appears still and readable.

Step 8 · Record smooth videos

A normal screen recording of this site stutters: the mouse wheel moves in steps, and heavy frames take longer than 1/60 of a second to draw. The fix is a recorder that controls time itself. It draws one exact frame, saves it, then moves the clock forward by exactly 1/60 of a second, so the video is perfectly smooth however slow the computer is.



Launch film - the site inside a browser, 40 s | Launch film - the end card | Website flow - full screen, 100 s

How the recorder (`flow.mjs`) works:

1. Puppeteer opens the site in a hidden Chrome window at 1920 × 1080.
2. Before the page loads, it swaps the page's clock (time, animation frames and timers) for a virtual one that moves only when the recorder says so.
3. For every frame it scrolls to the planned position, moves the clock forward 1/60 of a second, jumps each visible video to its matching frame, and saves a screenshot.
4. The scroll path is planned on the site's own scenes: slower while an arch opens and through the awakening, a short hold on each big moment, and one smooth curve with no jerks or overshoot.
5. ffmpeg joins the screenshots into the finished video.

```
# 1. all-keyframe copies of the clips, so the recorder can jump to any frame instantly
ffmpeg -i website/assets/video/hero.mp4 -c:v libx264 -g 1 -crf 18 -an intra/hero.mp4

# 2. the recording server (port 5174) serves the site using those copies
node server.js

# 3. record the whole site: about 100 seconds, 6,032 frames
node flow.mjs out/flow

# 4. join the frames into a 60 fps video
ffmpeg -framerate 60 -i out/flow/%05d.jpg -c:v libx264 -preset slow -crf 17 -pix_fmt yuv420p -movflags
+faststart website-flow.mp4
```

The soundtrack is built from scratch by `synth_flow.py` with numpy and scipy: kick, clap and hi-hats, an 808 bass line, an arpeggio, deep brass-like hits, risers and snare rolls, at 128 beats per minute. Its hit times come from the recording itself: a scan of every frame's brightness shows exactly when each arch fills the screen. It is mastered to about −12 LUFS (normal loudness for social media) and added without re-encoding the picture:

```
python synth_flow.py
ffmpeg -i website-flow-silent.mp4 -i flow_audio.wav -map 0:v -map 1:a -c:v copy -af
"volume=3.4dB,alimiter=limit=0.79" -c:a aac -b:a 320k website-flow.mp4
```

The 40-second launch film uses the same frame-by-frame idea on a separate page: a GSAP timeline that places the website recordings inside a browser window and a phone, adds titles, and is saved frame by frame by `render.mjs` . Every script is kept next to its video, in `website-flow/source` and `launch-video/source` .

Problems you might hit

Every problem below actually came up while building this site; each has a tested fix.

Problem	Why it happens	Fix
Clips freeze or will not loop	The page was opened straight from the folder, which cannot answer range requests	Run `node serve.js` and use http://localhost:5173
A picture appears sideways	It is landscape art saved sideways from a phone	Rotate it: `-vf transpose=2` (left) or `transpose=1` (right)
Subtitles show in a clip	The fan edit burned them into the picture	Crop a wide band above them (Step 3)
A loop blinks black or white every few seconds	The shot starts or ends on the source video's own flash or dissolve	Trim the start or end and check the first and last frames
Some black frames in the site videos	A clip was not rendered, or its poster is missing	Check every `data-src` and `poster` path exists in `assets/video`
A centred element jumps off-centre once animated	GSAP converts CSS `translate` centring into fixed pixels, sometimes before an image has loaded	Centre it with `inset: 0; margin: auto` instead
Two lines of text run together, as in "isstill"	Text split into lines inside a flex box lays the lines side by side	Split text inside a normal block element
A dark smudge around the loader's sun	The opening text's shadow appeared before the text	Fade the text and its shadow in together, once the film has opened
A horizontal chapter moves faster than the scroll	The pin was shorter than the animation	Make the pin length equal the timeline's duration
A screen recording stutters	The wheel scrolls in steps and frames take too long to draw	Use the frame-by-frame recorder (Step 8)
The recorder captures frozen video	Normal clips cannot jump to an exact frame quickly	Record with all-keyframe copies (`-g 1`)
Gallery pictures show as white boxes in a background tab	Hidden tabs skip decoding pictures	Call `img.decode()` on them before the scene begins

Credits and fan-use note

This is an unofficial fan project, made for fun and never sold. ONE PIECE is © Eiichiro Oda / Shueisha and Toei Animation; the artwork and footage belong to their creators, and the YouTube edits belong to their channels. Keep any rebuild non-commercial and credit them the same way. The music in both videos is original, synthesized for this project. Built with GSAP, Lenis, ffmpeg and Google Fonts.